

DataTypesAsLattices

Lean 4 formalization

ScottLean4 · Dana Scott formalization project

```
1 /-
2   Data Types as Lattices – the  $P\omega$  model (Lean 4 formalization, pilot)
3
4   Faithful to:
5     D. Scott, "Data Types as Lattices",
6     SIAM Journal on Computing 5(3):522–587, 1976.
7
8   Source text extracted from:
9     DanaScottPapers/Scott-1976-Data-Types-as-Lattices.pdf
10
11  This file transcribes the core objects of Section 1 ("Continuous functions"):
12    * The domain  $P\omega$  = the set of all subsets of  $\omega$  (the nonnegative integers),
13      a complete lattice under set inclusion  $\subseteq$ .
14    * Finite sets  $e_n$  enumerated by  $n$ , and the neighborhood basis.
15    * Continuity of  $f : P\omega \rightarrow P\omega$  (Theorem 1.1, the characterization theorem).
16    * The standard pairing  $\langle n, m \rangle = \frac{1}{2}(n+m)(n+m+1) + m$ .
17    *  $\text{graph}(f)$  and  $\text{fun}(u)$ , and the graph theorem (Theorem 1.2).
18    * Application  $u \cdot x = \text{fun}(u)(x)$ , the reflexive-domain operation.
19    * The fixed-point theorem (Theorem 1.4).
20
21  Core Lean 4 only; no Mathlib.  $P\omega$  is modelled as  $\text{'Nat} \rightarrow \text{Prop}'$  (a predicate on  $\omega$ ,
22  i.e. a subset of  $\omega$ );  $\subseteq$  is pointwise implication. We keep  $e_n$  and the
23  finite-subset relation abstract via classical-style predicates where a decision
24  procedure is not needed for the statements.
25  -/
26
27  namespace DataTypesAsLattices
28
29  /--  $P\omega$ : an element of Scott's domain is a subset of  $\omega$ , i.e. a set of integers,
30      represented by its characteristic predicate. -/
31  abbrev P $\omega$  := Nat  $\rightarrow$  Prop
32
33  /-- Set inclusion  $\subseteq$  – the partial order making  $P\omega$  a complete lattice. -/
34  def sub (x y : P $\omega$ ) : Prop :=  $\forall n, x\ n \rightarrow y\ n$ 
35
36  infix:50 "  $\subseteq$  " => sub
37
38  /-- Union (least upper bound of two elements). -/
39  def union (x y : P $\omega$ ) : P $\omega$  := fun n => x n  $\vee$  y n
40
41  /-- Bottom element: the empty set. -/
42  def bot : P $\omega$  := fun _ => False
43
44  /-! ## Finite subsets and the neighborhood basis
45
46      "A basis for the neighborhoods of  $P\omega$  consists of those sets of the form
47       $\{ x \in P\omega \mid e_n \subseteq x \}$  for a given  $e_n$ ."
48
```

```

49   `e_n` is the finite set of integers coded by `n`. We keep the specific coding
50   abstract as `enum : Nat → Pω`, together with the hypothesis (proved in the
51   paper for the standard coding) that every `enum n` is a *finite* subset. The
52   predicate `IsFinite` records finiteness abstractly. -/
53   axiom enum : Nat → Pω          -- e_n, the n-th finite set of integers
54
55   /-- `x` contains the finite set `e_n`. -/
56   def containsFin (n : Nat) (x : Pω) : Prop := enum n ⊆ x
57
58   /-! ## Continuity (Theorem 1.1, the characterization theorem)
59
60   "A function f : Pω → Pω is continuous iff for all x ∈ Pω and all e (= integer
61   m) we have:  m ∈ f(x)  iff  ∃ e_n ⊆ x. m ∈ f(e_n)."
62
63   This is the ε-δ (finite-approximation) form Scott gives as the most convenient
64   for proofs. -/
65   def Continuous (f : Pω → Pω) : Prop :=
66     ∀ (x : Pω) (m : Nat), (f x) m ↔ ∃ n, enum n ⊆ x ∧ (f (enum n)) m
67
68   /-- Continuous functions are monotone (the "positive character" of the topology):
69   "whenever x ⊆ y then f(x) ⊆ f(y)." -/
70   theorem Continuous.mono {f : Pω → Pω} (hf : Continuous f) :
71     ∀ {x y : Pω}, x ⊆ y → f x ⊆ f y := by
72     intro x y hxy m hm
73     rw [hf y m]
74     rw [hf x m] at hm
75     obtain ⟨n, hn, hfm⟩ := hm
76     exact ⟨n, fun k hk => hxy k (hn k hk), hfm⟩
77
78   /-! ## The standard enumeration of pairs
79
80   "(n, m) = ½(n + m)(n + m + 1) + m." -/
81   def pair (n m : Nat) : Nat := (n + m) * (n + m + 1) / 2 + m
82
83   /-! ## graph and fun (the reflexive structure)
84
85   "graph(f) = { (n, m) | m ∈ f(e_n) }"
86   "fun(u)(x) = { m | ∃ e_n ⊆ x. (n, m) ∈ u }"
87
88   `graph` turns a continuous function into an element of `Pω`; `apply` (Scott's
89   `fun`) turns an element of `Pω` into a function `Pω → Pω`. Their interplay is
90   the graph theorem (Theorem 1.2). -/
91   def graph (f : Pω → Pω) : Pω :=
92     fun k => ∃ n m, k = pair n m ∧ (f (enum n)) m
93
94   /-- `fun(u)` : the continuous function coded by the set `u`. This is Scott's
95   fundamental *application* operation on the reflexive domain `Pω`. -/
96   def apply (u : Pω) (x : Pω) : Pω :=
97     fun m => ∃ n, enum n ⊆ x ∧ u (pair n m)
98
99   /-- Scott's application infix `(u · x = fun(u)(x))`, making `Pω` a model of the
100   type-free λ-calculus / combinatory logic. -/
101   infixl:70 " · " => apply
102
103   /-- The function coded by any set `u` is continuous (part of Theorem 1.2 /
104   the graph theorem): `fun(u)` is a continuous function of `x`. -/
105   theorem apply_continuous (u : Pω) : Continuous (apply u) := by
106     intro x m
107     constructor
108     · intro h

```

```

109   obtain ⟨n, hn, hu⟩ := h
110   exact ⟨n, hn, ⟨n, fun k hk => hk, hu⟩⟩
111   · intro h
112   obtain ⟨n, hn, hm⟩ := h
113   obtain ⟨n', hn', hu⟩ := hm
114   exact ⟨n', fun k hk => hn k (hn' k hk), hu⟩
115
116 /-! ## Theorem 1.2 (the graph theorem)
117
118 "(i) fun(graph(f)) = f
119 (ii) u ⊆ graph(fun(u)), with equality iff u is closed under:
120 whenever (k, m) ∈ u and e_k ⊆ e_n, then (n, m) ∈ u."
121
122 We record (i) as the target statement `graph_theorem_i`. A full proof needs
123 the injectivity/monotonicity facts of the pairing and `enum` codings; here we
124 give the faithful signature Scott proves in the Appendix. -/
125 def graph_theorem_i : Prop :=
126   ∀ (f : Pω → Pω), Continuous f → ∀ x, (apply (graph f) x) = f x
127
128 /-! ## Theorem 1.4 (the fixed-point theorem)
129
130 "Every continuous function f : Pω → Pω has a least fixed point given by
131   fix(f) = ⋂ f^n(∅),
132   where ∅ is the empty set and f^n is the n-fold composition of f with itself."
133
134 `iterate f n bot` is f^n(∅); `fixApprox f n` is that n-th approximant, and
135 `fix f` is their union (supremum). -/
136 def iterate (f : Pω → Pω) : Nat → Pω → Pω
137 | 0, x => x
138 | k + 1, x => f (iterate f k x)
139
140 def fix (f : Pω → Pω) : Pω :=
141   fun m => ∃ n, (iterate f n bot) m
142
143 /- `fix f` is a fixed point of a continuous `f` (Theorem 1.4). -/
144 def fix_is_fixedpoint : Prop :=
145   ∀ (f : Pω → Pω), Continuous f → fix f = f (fix f)
146
147 /- `fix f` is the *least* fixed point (Theorem 1.4). -/
148 def fix_is_least : Prop :=
149   ∀ (f : Pω → Pω) (y : Pω), f y ⊆ y → fix f ⊆ y
150
151 end DataTypesAsLattices

```